

SATQUERY AI

P4 — Kevin

COMPLETE ENGINEERING PRD

Geospatial / Data Engineer

Deadline: 20 September 2026 • Six developers in parallel • Contract-first microservices

This is an execution document, not an API-only specification. It defines ownership, repository locations, modules, issues, dependencies, tests, security, observability, daily execution and a ready-to-copy coding-agent prompt.

1. Role Mission

Field	Implementation specification
Primary owner	Bhoonidhi/STAC, normalization, COG, GDAL/Rasterio, PostGIS, TiTiler
Backup/review	Monitoring data pipeline
Architecture reviewer	P1
QA/DevOps	P6
Success condition	Owned subsystem is independently runnable, contract-conformant, observable, secure and integrated into the flagship flood mission.
Non-ownership	Other teams' private tables, internal implementation details, credentials and undocumented contract changes.

2. System Position

P4 system position



Communication: browser→Gateway uses HTTPS/WebSocket; service-to-service uses versioned APIs/events; long-running EO/ML/report/monitoring work is asynchronous; storage and schemas have explicit ownership.

3. Repository / Folder Ownership

Folder	Purpose	Must not contain
services/eo-data/	Owned implementation, tests or docs	Unrelated service logic / credentials
services/geo/	Owned implementation, tests or docs	Unrelated service logic / credentials
data/ingestion/	Owned implementation, tests or docs	Unrelated service logic / credentials
data/stac/	Owned implementation, tests or docs	Unrelated service logic / credentials
data/preprocessing/	Owned implementation, tests or docs	Unrelated service logic / credentials
data/fixtures/	Owned implementation, tests or docs	Unrelated service logic / credentials

4. Technology Stack

Field	Implementation specification
Stack	Python/FastAPI/PySTAC/GDAL/Rasterio/GeoPandas/Shapely/PostGIS/S3-MinIO/TiTiler/Redis-Celery
Why	Matches the supplied SatQuery architecture and the six-person playbook; additions are implementation recommendations, not product requirements.
Serialization	OpenAPI/JSON Schema; canonical DTOs/events.
Testing	pytest and contract/integration tests; frontend uses browser E2E where applicable.
Observability	OpenTelemetry + structured logs + Prometheus/Grafana.
Containerization	Docker; Kubernetes remains future/post-SIH.

5. Engineering Rules

- All internal APIs use /api/v1; breaking changes require versioning and P1 review.
- Canonical ErrorResponse: code, message, details[], retryable, trace_id.
- Propagate trace_id, mission_id, run_id, job_id, organization_id where applicable.
- organization_id comes from verified auth context, never a request body.
- No cross-service private DB reads.
- Long-running EO/ML/report/monitoring work returns 202 + job_id.
- Results retain dataset_id/observation_ids/model_version/processing_version as applicable.
- Idempotency-Key is used for duplicate-sensitive writes.

- Remote calls use bounded timeouts/retries/backoff; no retry storms.
- Never log secrets, auth headers or raw sensitive inputs.

6. Complete Issue Register

ID	Title	Pri	Type	Depends	Contact
P4-01	EO/Data service skeleton and provider adapter interface	P0	Infrastructure	P1-01	P1
P4-02	Provider configuration and secret boundary	P0	Security	P1-01,P1-02	P3
P4-03	Observation and AssetRef domain model	P0	Data	P1-01,P1-02	P2
P4-04	STAC adapter	P0	Feature	P4-03	P1
P4-05	Bhoonidhi adapter	P0	Feature	P4-04	P3
P4-06	Observation normalization pipeline	P0	Data	P4-05	P2
P4-07	Spatial/temporal observation search	P0	Feature	P4-06,P1-02	P1
P4-08	Asset resolver	P0	Feature	P4-07,P1-02	P3
P4-09	Secure asset retrieval and content validation	P0	Security	P4-08,P1-02	P2
P4-10	Raster validation	P0	Geo/Security	P4-09,P1-02	P1
P4-11	CRS normalization and reprojection	P0	Geo	P4-10	P3
P4-12	AOI clipping and windowed processing	P0	Geo	P4-11	P2
P4-13	COG generation and overviews	P0	Geo	P4-12	P1
P4-14	PostGIS spatial operations	P0	Geo	P4-13	P3
P4-15	TiTiler integration	P1	Feature	P4-14,P3-05	P2
P4-16	Async GeoJob worker	P0	Infrastructure	P4-15,P3-05	P1
P4-17	Geo failure recovery and fixture fallback	P0	Reliability	P4-16,P3-05	P3
P4-18	Monitoring observation selection support	P1	Integration	P4-17,P3-05	P2
P4-19	EO data quality scoring/enrichment	P1	Feature	P4-18,P3-05	P1
P4-20	Pinned EO/Geo fixture pack and release hardening	P0	Release	P4-19,P3-05	P3

P4-01 — EO/Data service skeleton and provider adapter interface

Field	Implementation specification
Priority	P0
Type	Infrastructure
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement eo/data service skeleton and provider adapter interface as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for eo/data service skeleton and provider adapter interface. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_eo_data_service_skeleton_and_provider_adapter_interface_valid()', 'test_eo_data_service_skeleton_and_provider_adapter_interface_invalid_input()']
Integration tests	['test_p4_01_service_boundary()']
Contract tests	['test_p4_01_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, eo/data service skeleton and provider adapter interface completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-02
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run eo/data service skeleton and provider adapter interface against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-02 — Provider configuration and secret boundary

Field	Implementation specification
Priority	P0
Type	Security
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement provider configuration and secret boundary as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for provider configuration and secret boundary. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/Internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_provider_configuration_and_secret_boundary_valid()', 'test_provider_configuration_and_secret_boundary_invalid_input()']
Integration tests	['test_p4_02_service_boundary()']
Contract tests	['test_p4_02_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, provider configuration and secret boundary completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-03
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run provider configuration and secret boundary against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-03 — Observation and AssetRef domain model

Field	Implementation specification
Priority	P0
Type	Data
Owner	P4
Reviewer	P1
Cross-service contact	P2
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement observation and assetref domain model as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for observation and assetref domain model. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_observation_and_assetref_domain_model_valid()', 'test_observation_and_assetref_domain_model_invalid_input()']
Integration tests	['test_p4_03_service_boundary()']
Contract tests	['test_p4_03_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, observation and assetref domain model completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P1-01,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-04
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run observation and assetref domain model against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-04 — STAC adapter

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement stac adapter as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for stac adapter. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_stac_adapter_valid()', 'test_stac_adapter_invalid_input()']
Integration tests	['test_p4_04_service_boundary()']
Contract tests	['test_p4_04_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, stac adapter completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-03
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-05
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run stac adapter against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-05 — Bhoonidhi adapter

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement bhoonidhi adapter as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for bhoonidhi adapter. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_bhoonidhi_adapter_valid()', 'test_bhoonidhi_adapter_invalid_input()']
Integration tests	['test_p4_05_service_boundary()']
Contract tests	['test_p4_05_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, bhoonidhi adapter completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-04
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-06
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run bhoonidhi adapter against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-06 — Observation normalization pipeline

Field	Implementation specification
Priority	P0
Type	Data
Owner	P4
Reviewer	P1
Cross-service contact	P2
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement observation normalization pipeline as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for observation normalization pipeline. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_observation_normalization_pipeline_valid()', 'test_observation_normalization_pipeline_invalid_input()']
Integration tests	['test_p4_06_service_boundary()']
Contract tests	['test_p4_06_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, observation normalization pipeline completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-07
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run observation normalization pipeline against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-07 — Spatial/temporal observation search

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement spatial/temporal observation search as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for spatial/temporal observation search. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_spatial_temporal_observation_search_valid()', 'test_spatial_temporal_observation_search_invalid_input()']
Integration tests	['test_p4_07_service_boundary()']
Contract tests	['test_p4_07_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, spatial/temporal observation search completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-06,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-08
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run spatial/temporal observation search against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-08 — Asset resolver

Field	Implementation specification
Priority	P0
Type	Feature
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement asset resolver as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for asset resolver. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_asset_resolver_valid()', 'test_asset_resolver_invalid_input()']
Integration tests	['test_p4_08_service_boundary()']
Contract tests	['test_p4_08_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, asset resolver completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-07, P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-09
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run asset resolver against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-09 — Secure asset retrieval and content validation

Field	Implementation specification
Priority	P0
Type	Security
Owner	P4
Reviewer	P1
Cross-service contact	P2
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement secure asset retrieval and content validation as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for secure asset retrieval and content validation. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_secure_asset_retrieval_and_content_validation_valid()', 'test_secure_asset_retrieval_and_content_validation_invalid_input()']
Integration tests	['test_p4_09_service_boundary()']
Contract tests	['test_p4_09_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, secure asset retrieval and content validation completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-08,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-10
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run secure asset retrieval and content validation against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-10 — Raster validation

Field	Implementation specification
Priority	P0
Type	Geo/Security
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement raster validation as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for raster validation. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_raster_validation_valid()', 'test_raster_validation_invalid_input()']
Integration tests	['test_p4_10_service_boundary()']
Contract tests	['test_p4_10_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, raster validation completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-09,P1-02
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-11
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run raster validation against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-11 — CRS normalization and reprojection

Field	Implementation specification
Priority	P0
Type	Geo
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement crs normalization and reprojection as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for crs normalization and reprojection. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_crs_normalization_and_reprojection_valid()', 'test_crs_normalization_and_reprojection_invalid_input()']
Integration tests	['test_p4_11_service_boundary()']
Contract tests	['test_p4_11_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, crs normalization and reprojection completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-10
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-12
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run crs normalization and reprojection against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-12 — AOI clipping and windowed processing

Field	Implementation specification
Priority	P0
Type	Geo
Owner	P4
Reviewer	P1
Cross-service contact	P2
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement aoi clipping and windowed processing as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for aoi clipping and windowed processing. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_aoi_clipping_and_windowed_processing_valid()', 'test_aoi_clipping_and_windowed_processing_invalid_input()']
Integration tests	['test_p4_12_service_boundary()']
Contract tests	['test_p4_12_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, aoi clipping and windowed processing completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-11
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-13
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run aoi clipping and windowed processing against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-13 — COG generation and overviews

Field	Implementation specification
Priority	P0
Type	Geo
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement cog generation and overviews as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for cog generation and overviews. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_cog_generation_and_overviews_valid()', 'test_cog_generation_and_overviews_invalid_input()']
Integration tests	['test_p4_13_service_boundary()']
Contract tests	['test_p4_13_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, cog generation and overviews completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-12
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-14
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run cog generation and overviews against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-14 — PostGIS spatial operations

Field	Implementation specification
Priority	P0
Type	Geo
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement postgis spatial operations as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for postgis spatial operations. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_postgis_spatial_operations_valid()', 'test_postgis_spatial_operations_invalid_input()']
Integration tests	['test_p4_14_service_boundary()']
Contract tests	['test_p4_14_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, postgis spatial operations completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-13
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-15
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run postgis spatial operations against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-15 — TiTiler integration

Field	Implementation specification
Priority	P1
Type	Feature
Owner	P4
Reviewer	P1
Cross-service contact	P2
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement titiler integration as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for titiler integration. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_titiler_integration_valid()', 'test_titiler_integration_invalid_input()']
Integration tests	['test_p4_15_service_boundary()']
Contract tests	['test_p4_15_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, titiler integration completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-14,P3-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-16
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run titiler integration against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-16 — Async GeoJob worker

Field	Implementation specification
Priority	P0
Type	Infrastructure
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement async geojob worker as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for async geojob worker. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_async_geojob_worker_valid()', 'test_async_geojob_worker_invalid_input()']
Integration tests	['test_p4_16_service_boundary()']
Contract tests	['test_p4_16_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, async geojob worker completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-15,P3-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-17
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run async geojob worker against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-17 — Geo failure recovery and fixture fallback

Field	Implementation specification
Priority	P0
Type	Reliability
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement geo failure recovery and fixture fallback as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for geo failure recovery and fixture fallback. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_geo_failure_recovery_and_fixture_fallback_valid()', 'test_geo_failure_recovery_and_fixture_fallback_invalid_input()']
Integration tests	['test_p4_17_service_boundary()']
Contract tests	['test_p4_17_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, geo failure recovery and fixture fallback completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-16,P3-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-18
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run geo failure recovery and fixture fallback against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-18 — Monitoring observation selection support

Field	Implementation specification
Priority	P1
Type	Integration
Owner	P4
Reviewer	P1
Cross-service contact	P2
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement monitoring observation selection support as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for monitoring observation selection support. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_monitoring_observation_selection_support_valid()', 'test_monitoring_observation_selection_support_invalid_input()']
Integration tests	['test_p4_18_service_boundary()']
Contract tests	['test_p4_18_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, monitoring observation selection support completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-17,P3-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-19
Parallelizable with	P2 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P2. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run monitoring observation selection support against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-19 — EO data quality scoring/enrichment

Field	Implementation specification
Priority	P1
Type	Feature
Owner	P4
Reviewer	P1
Cross-service contact	P1
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement eo data quality scoring/enrichment as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for eo data quality scoring/enrichment. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_eo_data_quality_scoring_enrichment_valid()', 'test_eo_data_quality_scoring_enrichment_invalid_input()']
Integration tests	['test_p4_19_service_boundary()']
Contract tests	['test_p4_19_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, eo data quality scoring/enrichment completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-18,P3-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	P4-20
Parallelizable with	P1 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P1. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run eo data quality scoring/enrichment against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

P4-20 — Pinned EO/Geo fixture pack and release hardening

Field	Implementation specification
Priority	P0
Type	Release
Owner	P4
Reviewer	P1
Cross-service contact	P3
Service	EO Data / Geo
Folders	services/eo-data/; services/geo/; data/ingestion/; data/stac/; data/preprocessing/; data/fixtures/
Objective	Implement pinned eo/geo fixture pack and release hardening as a production-shaped module inside the P4 ownership boundary, with deterministic behavior, explicit contracts, tests, security and telemetry.
Business value	Required to make the flagship natural-language → evidence → monitoring workflow reliable and independently integrable.
Technical scope	Implement the domain logic, adapters, validation, persistence/worker boundaries and integration hooks needed for pinned eo/geo fixture pack and release hardening. Keep provider/model-specific details behind the owned boundary.
Implementation steps	['Inspect current contracts and neighboring service interfaces.', 'Implement deterministic core logic first, then adapters/integration hooks.', 'Add schema/input validation and explicit error handling.', 'Add unit + contract + integration tests and telemetry.', 'Run formatter/linter/type checks and verify in DEV/DEMO.']
Files to create	services/eo-data/implementation.py; corresponding tests/unit and tests/integration fixture files
Files to modify	packages/contracts or service configuration only when the issue explicitly changes a contract; update docs/ADR when architecture changes.
Classes/modules	Domain/service module; validator; adapter/client where required; worker where asynchronous; telemetry hooks.
API	/api/v1/observations/search, /api/v1/assets/resolve, /api/v1/geo/jobs, /api/v1/tiles
Database/storage	PostgreSQL/PostGIS for owned structured state; S3/MinIO for EO-derived artifacts where applicable; never read another service private tables.
Queue/async	Redis/Celery or Redis Streams for long-running work/events where applicable; synchronous request only for lightweight validation/query.
External systems	Bhoonidhi/STAC for P4; model artifacts/inference runtime for P3; Gateway/internal services through allowlisted APIs for other roles.
Input	Canonical request/DTO, verified auth context, mission_id/run_id/job_id and trace_id where applicable.
Output	Schema-valid domain result, explicit status/error, provenance and correlation metadata.
Validation	Pydantic/OpenAPI schema validation; GeoJSON/raster validation for geo work; allowlists for URLs/tools; tenant authorization.
Error handling	Use canonical ErrorResponse. Map timeout/upstream failures explicitly; bounded retries only; never silently fabricate success.
Security	OAuth2/OIDC/JWT/RBAC/tenant isolation as applicable; least privilege; no secret logging; SSRF/file/raster/prompt/tool defenses where applicable.
Observability	Structured logs + OpenTelemetry spans; propagate trace_id, mission_id, run_id, job_id, organization_id; emit role-specific latency/error metrics.
Unit tests	['test_pinned_eo_geo_fixture_pack_and_release_hardening_valid()', 'test_pinned_eo_geo_fixture_pack_and_release_hardening_invalid_input()']
Integration tests	['test_p4_20_service_boundary()']
Contract tests	['test_p4_20_schema_compatibility()']
Performance tests	Benchmark if runtime-facing; compare against source NFRs (<500ms normal UI, <2s metadata search, <10s small analysis where feasible).
Acceptance criteria	Given valid canonical inputs, pinned eo/geo fixture pack and release hardening completes with the expected structured output, preserves correlation metadata and exposes explicit failure states.
Definition of Done	Merged via PR review; unit/contract/integration tests green; security checks pass; telemetry exists; docs updated; DEV/DEMO verified; acceptance demonstrated.
Dependencies	P4-19,P3-05
Blocked by	None initially; use the dependency issue or its contract fixture if implementation is unfinished.
Blocks	Release/integration gate
Parallelizable with	P3 domain work where the frozen contract is sufficient; P6 CI work is always parallelizable.
Cross-team communication	Primary: P4. Consumer/contact: P3. Contract authority: P1. CI/release: P6. Communicate schema, failure semantics, fixtures and demo evidence.
Demo verification	Run pinned eo/geo fixture pack and release hardening against the pinned relevant fixture and show the resulting API/state/telemetry in the integrated mission flow.
Rollback	Revert the feature commit or disable the explicit feature/profile flag; preserve prior compatible contract and deterministic fixture path.

7. Test Plan

The issue-level test names below are the minimum required suite. Expand edge cases discovered during implementation.

P4-01 — EO/Data service skeleton and provider adapter interface

- test_eo_data_service_skeleton_and_provider_adapter_interface_valid()
- test_eo_data_service_skeleton_and_provider_adapter_interface_invalid_input()
- test_p4_01_service_boundary()
- test_p4_01_schema_compatibility()

P4-02 — Provider configuration and secret boundary

- test_provider_configuration_and_secret_boundary_valid()
- test_provider_configuration_and_secret_boundary_invalid_input()
- test_p4_02_service_boundary()
- test_p4_02_schema_compatibility()

P4-03 — Observation and AssetRef domain model

- test_observation_and_assetref_domain_model_valid()
- test_observation_and_assetref_domain_model_invalid_input()
- test_p4_03_service_boundary()
- test_p4_03_schema_compatibility()

P4-04 — STAC adapter

- test_stac_adapter_valid()
- test_stac_adapter_invalid_input()
- test_p4_04_service_boundary()
- test_p4_04_schema_compatibility()

P4-05 — Bhoonidhi adapter

- test_bhoonidhi_adapter_valid()
- test_bhoonidhi_adapter_invalid_input()
- test_p4_05_service_boundary()
- test_p4_05_schema_compatibility()

P4-06 — Observation normalization pipeline

- test_observation_normalization_pipeline_valid()
- test_observation_normalization_pipeline_invalid_input()
- test_p4_06_service_boundary()
- test_p4_06_schema_compatibility()

P4-07 — Spatial/temporal observation search

- test_spatial_temporal_observation_search_valid()
- test_spatial_temporal_observation_search_invalid_input()
- test_p4_07_service_boundary()
- test_p4_07_schema_compatibility()

P4-08 — Asset resolver

- test_asset_resolver_valid()
- test_asset_resolver_invalid_input()
- test_p4_08_service_boundary()
- test_p4_08_schema_compatibility()

P4-09 — Secure asset retrieval and content validation

- `test_secure_asset_retrieval_and_content_validation_valid()`
- `test_secure_asset_retrieval_and_content_validation_invalid_input()`
- `test_p4_09_service_boundary()`
- `test_p4_09_schema_compatibility()`

P4-10 — Raster validation

- `test_raster_validation_valid()`
- `test_raster_validation_invalid_input()`
- `test_p4_10_service_boundary()`
- `test_p4_10_schema_compatibility()`

P4-11 — CRS normalization and reprojection

- `test_crs_normalization_and_reprojection_valid()`
- `test_crs_normalization_and_reprojection_invalid_input()`
- `test_p4_11_service_boundary()`
- `test_p4_11_schema_compatibility()`

P4-12 — AOI clipping and windowed processing

- `test_aoi_clipping_and_windowed_processing_valid()`
- `test_aoi_clipping_and_windowed_processing_invalid_input()`
- `test_p4_12_service_boundary()`
- `test_p4_12_schema_compatibility()`

P4-13 — COG generation and overviews

- `test_cog_generation_and_overviews_valid()`
- `test_cog_generation_and_overviews_invalid_input()`
- `test_p4_13_service_boundary()`
- `test_p4_13_schema_compatibility()`

P4-14 — PostGIS spatial operations

- `test_postgis_spatial_operations_valid()`
- `test_postgis_spatial_operations_invalid_input()`
- `test_p4_14_service_boundary()`
- `test_p4_14_schema_compatibility()`

P4-15 — TiTiler integration

- `test_titiler_integration_valid()`
- `test_titiler_integration_invalid_input()`
- `test_p4_15_service_boundary()`
- `test_p4_15_schema_compatibility()`

P4-16 — Async GeoJob worker

- `test_async_geojob_worker_valid()`
- `test_async_geojob_worker_invalid_input()`
- `test_p4_16_service_boundary()`
- `test_p4_16_schema_compatibility()`

P4-17 — Geo failure recovery and fixture fallback

- `test_geo_failure_recovery_and_fixture_fallback_valid()`
- `test_geo_failure_recovery_and_fixture_fallback_invalid_input()`
- `test_p4_17_service_boundary()`
- `test_p4_17_schema_compatibility()`

P4-18 — Monitoring observation selection support

- test_monitoring_observation_selection_support_valid()
- test_monitoring_observation_selection_support_invalid_input()
- test_p4_18_service_boundary()
- test_p4_18_schema_compatibility()

P4-19 — EO data quality scoring/enrichment

- test_eo_data_quality_scoring_enrichment_valid()
- test_eo_data_quality_scoring_enrichment_invalid_input()
- test_p4_19_service_boundary()
- test_p4_19_schema_compatibility()

P4-20 — Pinned EO/Geo fixture pack and release hardening

- test_pinned_eo_geo_fixture_pack_and_release_hardening_valid()
- test_pinned_eo_geo_fixture_pack_and_release_hardening_invalid_input()
- test_p4_20_service_boundary()
- test_p4_20_schema_compatibility()

8. Security Implementation

Control	Implementation owner	Required behavior
Authentication	P4	Implement/test where applicable; P6 supplies regression gate.
RBAC + tenant isolation	P4	Implement/test where applicable; P6 supplies regression gate.
Input/schema validation	P4	Implement/test where applicable; P6 supplies regression gate.
GeoJSON/raster validation	P4	Implement/test where applicable; P6 supplies regression gate.
SSRF/provider allowlist	P4	Implement/test where applicable; P6 supplies regression gate.
Prompt injection	P4	Implement/test where applicable; P6 supplies regression gate.
AI tool permissions	P4	Implement/test where applicable; P6 supplies regression gate.
Rate/request limits	P4	Implement/test where applicable; P6 supplies regression gate.
Secrets/TLS	P4	Implement/test where applicable; P6 supplies regression gate.
Audit logging	P4	Implement/test where applicable; P6 supplies regression gate.
Dependency/container scanning	P4	Implement/test where applicable; P6 supplies regression gate.

9. Observability

Required identifiers: trace_id, mission_id, run_id, job_id, organization_id, model_version, dataset_id. Metrics should expose latency, queue time, errors and user-visible mission state. Role metrics: geo_search_latency_ms, geo_job_duration_ms, asset_download_failure_total, raster_validation_failure_total

10. Failure Modes

Failure	Detection	Recovery/fallback	User behavior
Bhoonidhi unavailable	timeout/5xx	bounded retry → pinned permitted fixture	show fallback clearly
Inference timeout/OOM	job timeout/worker exit	bounded retry → deterministic fixture for demo	show explicit degraded/failure state
Redis restart	connection/requeue failure	idempotent requeue/reconciliation	mission remains recoverable
Invalid AOI/raster	validator rejection	no expensive processing	show actionable validation
Low confidence	confidence gate	clarify/acquire/explicit uncertainty	never confident prose
Tenant violation	RBAC denial	no retry/no leakage	403/404

11. Implementation Order

Foundation → core domain → cross-service integration → differentiation → hardening/release. Earlier issues unlock later ones, but frozen contracts and fixtures allow parallel implementation. Do not serialize on live dependencies. Use mocks whenever the producer is unfinished.

12. Daily Execution Plan — Sep 11–20

Date	Issues	Expected output	Dependency	Integration checkpoint	Tests/demo
Sep 11	P4-01, P4-02	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 12	P4-02, P4-03	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 13	P4-03, P4-04	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 14	P4-04, P4-05	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 15	P4-05, P4-06	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 16	P4-06, P4-07	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 17	P4-07, P4-08	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 18	P4-08, P4-09	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 19	P4-09, P4-10	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state
Sep 20	P4-10, P4-11	Working code + fixture	Named dependency in issue DAG	Daily shared DEV checkpoint	Unit/contract + demo state

13. Definition of Ready / Done

Field	Implementation specification
DoR	Requirements clear; owner/reviewer; contract; dependencies; acceptance; tests; demo result defined.
DoD	Merged; unit/contract/integration tests; security; telemetry; docs/schema; DEV/DEMO; acceptance demonstrated.

14. Cross-Team Communication

- P1 is architecture/contract authority.
- P6 is CI/DEV/DEMO/release authority.
- P2↔P3: sensor arbitration, inference and evidence.
- P2↔P4: observation selection, data quality and acquisition.
- P3↔P4: AssetRef, CRS/resolution and preprocessing.
- P4↔P5: tiles, masks, GeoJSON and map performance.
- P5↔P1: generated client and browser DTOs.
- Escalate blockers after 30–60 minutes; use contract mocks/fixtures instead of waiting.

15. Prompt to Give to the Coding Agent

You are P4 implementation agent. Own E0 Data and Geo Processing. Keep Bhoonidhi/STAC behind provider-neutral adapters. Canonicalize Observation/AssetRef. Validate GeoJSON/raster/CRS/content; defend against SSRF, path traversal and malicious files. Use PostGIS, S3/MinIO, COG, GDAL/Rasterio, TiTiler and Redis/Celery as applicable. Clip early and window large rasters. Add geo correctness, contract, security and integration tests plus pinned fixtures.

16. Role Release Checklist

- [] All P0 issues merged or explicitly waived.
- [] Unit + contract + relevant integration tests green.
- [] Security controls and negative tests green.
- [] Trace/metrics/logging present.
- [] DEV/DEMO verified.
- [] Failure behavior demonstrated.
- [] Fixtures/model versions pinned where applicable.
- [] Documentation/ADR updated.
- [] Role-specific flagship demo path works.